

CSS if() Function

Complete Guide: Easy English Edition

Master conditional logic in CSS with the `if()` function

Contents

1	Introduction	2
1.1	What is the if() Function?	2
1.2	Why Use if()?	2
2	if() Function Basics	3
2.1	Basic Syntax	3
2.2	Simple Example	3
3	Comparison Conditions	4
3.1	Available Operators	4
3.2	Size Comparisons	4
4	Logical Operators	5
4.1	AND Operator	5
4.2	OR Operator	5
4.3	NOT Operator	5
4.4	Complex Conditions	5
5	Media Query Replacement	7
5.1	Before: Media Queries	7
5.2	After: if() Function	7
6	Real-World Examples	8
6.1	Example 1: Responsive Grid	8
6.2	Example 2: Conditional Layouts	8
6.3	Example 3: Mobile-First Typography	8
6.4	Example 4: Navigation Menu	9
7	Container Queries with if()	10
7.1	Container-Based Conditions	10
7.2	Nested Container Logic	10
8	Aspect Ratio Conditions	11
8.1	Based on Aspect Ratio	11
8.2	Responsive Image Handling	11
9	Color and Theme Conditions	12
9.1	Prefers Dark Mode	12
9.2	Prefers Reduced Motion	12
10	Browser Support	13
10.1	Current Status	13
10.2	Fallback Strategy	13

11 Comparison with Alternatives	14
11.1 if() vs. Media Queries	14
11.2 if() vs. Container Queries	14
12 Advanced Patterns	15
12.1 Nested if() Statements	15
12.2 Multiple Conditions	15
12.3 Combining with Custom Properties	15
13 Common Patterns	17
13.1 Pattern 1: Mobile-First Approach	17
13.2 Pattern 2: Responsive Typography	17
13.3 Pattern 3: Adaptive Spacing	17
14 Common Mistakes	18
14.1 Mistake 1: Missing Fallback Value	18
14.2 Mistake 2: Wrong Comparison Syntax	18
14.3 Mistake 3: No Browser Support Check	18
15 Quick Reference	20
15.1 Basic Syntax	20
15.2 Common Conditions	20
15.3 Key Points	20

1 Introduction

The CSS `if()` function lets you write conditional logic directly in CSS! Instead of using media queries for everything, you can now use simple if-conditions to control your styles.

1.1 What is the `if()` Function?

The `if()` function allows you to:

- **Conditional Styling:** Apply styles based on conditions
- **Simpler Logic:** No more complex media queries
- **Dynamic Styling:** Respond to various conditions
- **Cleaner Code:** More readable CSS

1.2 Why Use `if()`?

- **Cleaner Syntax:** More intuitive than media queries
- **Less Repetition:** Avoid duplicating selectors
- **Better Logic:** Complex conditions easier to express
- **Maintainability:** Easier to understand intent
- **Progressive Enhancement:** Graceful fallbacks

2 if() Function Basics

2.1 Basic Syntax

```
if(condition) {  
  /* Styles if condition is true */  
} else {  
  /* Styles if condition is false */  
}
```

Important: The CSS `if()` function is still experimental and not widely supported yet. This guide covers the proposed syntax!

2.2 Simple Example

```
/* If screen is wide, make layout horizontal */  
.layout {  
  display: if(  
    (width > 768px),  
    flex,  
    block  
  );  
  flex-direction: if(  
    (width > 768px),  
    row,  
    column  
  );  
}
```

3 Comparison Conditions

3.1 Available Operators

Operator	Meaning	Example
>	Greater than	width > 768px
<	Less than	height < 600px
>=	Greater or equal	width >= 768px
<=	Less or equal	height <= 600px
==	Equal	color == blue
!=	Not equal	color != red

3.2 Size Comparisons

```
/* Compare container width */
.responsive-box {
  font-size: if(
    (width > 1200px),
    24px,
    16px
  );
}

/* Compare height */
.tall-layout {
  justify-content: if(
    (height > 800px),
    space-around,
    space-between
  );
}

/* Compare aspect ratio */
.media {
  object-fit: if(
    (aspect-ratio > 1),
    cover,
    contain
  );
}
```

4 Logical Operators

4.1 AND Operator

Combine multiple conditions with AND:

```
.smart-layout {  
  display: if(  
    (width > 768px) and (height > 600px),  
    grid,  
    flex  
  );  
}
```

4.2 OR Operator

Check if any condition is true:

```
.mobile-safe {  
  padding: if(  
    (width < 480px) or (height < 600px),  
    12px,  
    24px  
  );  
}
```

4.3 NOT Operator

Invert a condition:

```
.desktop-only {  
  display: if(  
    not (width < 768px),  
    block,  
    none  
  );  
}
```

4.4 Complex Conditions

```
.complex-responsive {  
  font-size: if(  
    ((width > 1200px) and (height > 800px)) or (aspect-ratio > 2),  
    32px,  
    16px  
  );  
}
```


}

5 Media Query Replacement

5.1 Before: Media Queries

```
.box {  
  font-size: 16px;  
  padding: 20px;  
}  
  
@media (min-width: 768px) {  
  .box {  
    font-size: 24px;  
    padding: 30px;  
  }  
}  
  
@media (min-width: 1200px) {  
  .box {  
    font-size: 32px;  
    padding: 40px;  
  }  
}
```

5.2 After: if() Function

```
.box {  
  font-size: if(  
    (width >= 1200px),  
    32px,  
    if(  
      (width >= 768px),  
      24px,  
      16px  
    )  
  );  
  
  padding: if(  
    (width >= 1200px),  
    40px,  
    if(  
      (width >= 768px),  
      30px,  
      20px  
    )  
  );  
}
```

Much cleaner!

6 Real-World Examples

6.1 Example 1: Responsive Grid

```
.grid {
  display: grid;
  grid-template-columns: if(
    (width >= 1200px),
    repeat(4, 1fr),
    if(
      (width >= 768px),
      repeat(2, 1fr),
      1fr
    )
  );
  gap: if(
    (width >= 768px),
    20px,
    12px
  );
}
```

6.2 Example 2: Conditional Layouts

```
.main-layout {
  display: if(
    (width > 768px),
    grid,
    flex
  );

  grid-template-columns: if(
    (width > 768px),
    250px 1fr 250px,
    1fr
  );

  flex-direction: if(
    (width > 768px),
    row,
    column
  );
}
```

6.3 Example 3: Mobile-First Typography

```
h1 {  
  font-size: if(  
    (width < 480px),  
    24px,  
    if(  
      (width < 768px),  
      32px,  
      48px  
    )  
  );  
  
  line-height: if(  
    (width < 768px),  
    1.3,  
    1.2  
  );  
}
```

6.4 Example 4: Navigation Menu

```
nav {  
  display: if(  
    (width >= 768px),  
    flex,  
    block  
  );  
  
  background: if(  
    (width >= 768px),  
    transparent,  
    #333  
  );  
}  
  
nav a {  
  display: if(  
    (width >= 768px),  
    inline-block,  
    block  
  );  
  
  padding: if(  
    (width >= 768px),  
    10px 20px,  
    15px  
  );  
}
```

7 Container Queries with if()

7.1 Container-Based Conditions

```
@container (min-width: 400px) {  
  .card {  
    display: if(  
      (width >= 400px),  
      grid,  
      flex  
    );  
  
    grid-template-columns: if(  
      (width >= 500px),  
      1fr 1fr,  
      1fr  
    );  
  }  
}
```

7.2 Nested Container Logic

```
.component {  
  padding: if(  
    (width > 600px) and (height > 400px),  
    30px,  
    if(  
      (width > 400px),  
      20px,  
      15px  
    )  
  );  
}
```

8 Aspect Ratio Conditions

8.1 Based on Aspect Ratio

```
.media-box {  
  /* Landscape (wider than tall) */  
  object-fit: if(  
    (aspect-ratio > 1),  
    cover,  
    contain  
  );  
  
  /* Very wide panorama */  
  margin: if(  
    (aspect-ratio > 2),  
    40px 0,  
    20px 0  
  );  
}
```

8.2 Responsive Image Handling

```
img {  
  width: if(  
    (aspect-ratio > 1.5),  
    100%,  
    if(  
      (aspect-ratio > 1),  
      80%,  
      60%  
    )  
  );  
  
  height: auto;  
}
```

9 Color and Theme Conditions

9.1 Prefers Dark Mode

```
.theme-aware {  
  background: if(  
    (prefers-color-scheme == dark),  
    #1e1e1e,  
    #ffffff  
  );  
  
  color: if(  
    (prefers-color-scheme == dark),  
    #ffffff,  
    #000000  
  );  
}
```

9.2 Prefers Reduced Motion

```
.animated-element {  
  animation: if(  
    (prefers-reduced-motion == reduce),  
    none,  
    slideIn 0.5s ease  
  );  
  
  transition: if(  
    (prefers-reduced-motion == reduce),  
    none,  
    all 0.3s ease  
  );  
}
```

10 Browser Support

10.1 Current Status

Browser	Support
Chrome	Experimental (behind flag)
Firefox	Experimental (behind flag)
Safari	Under consideration
Edge	Experimental (behind flag)

Note: The `if()` function is still in development. Use with caution and always provide fallbacks!

10.2 Fallback Strategy

```
/* Fallback for unsupported browsers */
.box {
  font-size: 16px;

  /* If supported, use if() */
  font-size: if(
    (width >= 768px),
    24px,
    16px
  );
}

/* Or use media queries as fallback */
.box {
  font-size: 16px;
}

@supports (font-size: if((width > 0px), 16px, 16px)) {
  .box {
    font-size: if(
      (width >= 768px),
      24px,
      16px
    );
  }
}
```


11 Comparison with Alternatives

11.1 if() vs. Media Queries

Media Queries:

```
@media (min-width: 768px) {  
  .box { font-size: 24px; }  
}
```

if() Function:

```
.box {  
  font-size: if((width >= 768px), 24px, 16px);  
}
```

11.2 if() vs. Container Queries

Container Queries:

```
@container (min-width: 400px) {  
  .card { display: grid; }  
}
```

if() with Containers:

```
.card {  
  display: if((width >= 400px), grid, flex);  
}
```

12 Advanced Patterns

12.1 Nested if() Statements

```
.responsive {  
  padding: if(  
    (width >= 1200px),  
    40px,  
    if(  
      (width >= 768px),  
      30px,  
      if(  
        (width >= 480px),  
        20px,  
        15px  
      )  
    )  
  );  
}
```

12.2 Multiple Conditions

```
.smart-component {  
  display: if(  
    ((width > 768px) and (height > 600px)) or (aspect-ratio > 1.5),  
    grid,  
    flex  
  );  
}
```

12.3 Combining with Custom Properties

```
:root {  
  --breakpoint-mobile: 480px;  
  --breakpoint-tablet: 768px;  
  --breakpoint-desktop: 1200px;  
}  
  
.responsive {  
  font-size: if(  
    (width >= var(--breakpoint-desktop)),  
    32px,  
    if(  
      (width >= var(--breakpoint-tablet)),  
      24px,  
      18px  
    )  
  );  
}
```

```
    16px  
    )  
    );  
}
```

13 Common Patterns

13.1 Pattern 1: Mobile-First Approach

```
.container {  
  display: flex;  
  flex-direction: column;  
  
  /* Override for larger screens */  
  flex-direction: if(  
    (width >= 768px),  
    row,  
    column  
  );  
}
```

13.2 Pattern 2: Responsive Typography

```
body {  
  font-size: if(  
    (width >= 1200px),  
    18px,  
    if(  
      (width >= 768px),  
      16px,  
      14px  
    )  
  );  
}
```

13.3 Pattern 3: Adaptive Spacing

```
.section {  
  margin-top: if(  
    (width >= 768px),  
    40px,  
    20px  
  );  
  
  margin-bottom: if(  
    (width >= 768px),  
    40px,  
    20px  
  );  
}
```

14 Common Mistakes

14.1 Mistake 1: Missing Fallback Value

DON'T:

```
.box {  
  font-size: if((width > 768px), 24px);  
  /* Incomplete! */  
}
```

DO:

```
.box {  
  font-size: if(  
    (width > 768px),  
    24px,  
    16px  
  );  
}
```

14.2 Mistake 2: Wrong Comparison Syntax

DON'T:

```
.box {  
  display: if(  
    width > 768px, /* Missing parentheses! */  
    flex,  
    block  
  );  
}
```

DO:

```
.box {  
  display: if(  
    (width > 768px),  
    flex,  
    block  
  );  
}
```

14.3 Mistake 3: No Browser Support Check

Problem: Using `if()` without fallbacks

Solution:

```
/* Always provide CSS fallback */  
.box {  
  display: flex; /* Fallback */  
  
  /* Enhanced for supporting browsers */  
  display: if((width > 768px), grid, flex);  
}
```

15 Quick Reference

15.1 Basic Syntax

```
/* Simple condition */
property: if(
  (condition),
  value-if-true,
  value-if-false
);

/* Nested conditions */
property: if(
  (condition1),
  value1,
  if(
    (condition2),
    value2,
    value3
  )
);

/* Multiple conditions */
property: if(
  (condition1) and (condition2),
  value1,
  value2
);
```

15.2 Common Conditions

```
(width > 768px)
(width >= 1200px)
(height < 600px)
(aspect-ratio > 1)
(prefers-color-scheme == dark)
(prefers-reduced-motion == reduce)
```

15.3 Key Points

- `if()` function provides conditional CSS styling
- Supports comparison operators: `>`, `<`, `>=`, `<=`, `==`, `!=`
- Combine conditions with `and`, `or`, `not`

- Can be nested for multiple breakpoints
- Still experimental - use with fallbacks
- Replace complex media queries
- Works with container queries

Conclusion

The CSS `if()` function represents the future of conditional styling in CSS. While still experimental, it offers a cleaner, more intuitive way to write responsive designs compared to media queries.

Key Takeaways:

- `if()` replaces complex media query logic
- Syntax: `if(condition, true-value, false-value)`
- Supports nested and complex conditions
- Always provide fallbacks for unsupported browsers
- Progressive enhancement keeps your styles working
- Monitor browser support as it evolves

Start experimenting with `if()` function for cleaner, more maintainable CSS!